
Pré-processamento e Índice Invertido

Aula 03 — Limpeza de texto, stopwords, stemming e o índice

Projeto Integrador III ■ Ciência de Dados

FATEC — Tecnologia em Ciência de Dados

Por que pré-processar o texto?

Texto real é “sujo”

- Maiúsculas/minúsculas
- Pontuação e acentos
- Palavras muito comuns
- Variações da mesma palavra

Objetivo de hoje

- Normalizar o texto
- Remover *stopwords*
- Reduzir ao radical (*stemming*)
- Construir um **índice invertido**

Ideia

Bom pré-processamento → vocabulário menor, índice mais limpo e busca melhor.

- **Caixa:** tudo para minúsculas (“Modelo” = “modelo”).
- **Pontuação:** remover símbolos que não carregam significado.
- **Acentos:** opcionalmente remover (`iconv(x, to="ASCII//TRANSLIT")`).
- **Espaços:** colapsar múltiplos espaços em um só.

Objetivo: que formas diferentes do “mesmo” termo colapsem em uma única chave.

O texto como ele chega: sujo

```
brutos <- c(  
  d1 = "Recuperacao de Informacao: ORDENA documentos, por relevancia!",  
  d2 = "O modelo de espaco-vetorial representa documentos (como vetores).",  
  d3 = "BM25 e um modelo probabilistico de ranqueamento de texto.",  
  d4 = "Aprendizado estatistico fundamenta a recuperacao moderna.",  
  d5 = "O indice invertido acelera a busca em muitos documentos.",  
  d6 = "Embeddings capturam a semantica de palavras e documentos.",  
  d7 = "A avaliacao mede a relevancia dos resultados da busca.",  
  d8 = "Ciencia de dados combina estatistica e programacao."  
)
```

Maiúsculas, dois-pontos, parênteses, hífen, ponto final, exclamação. Nada disso ajuda a busca.

Limpendo o texto em R

```
limpar <- function(x) {  
  x <- tolower(x)           # 1) tudo minuscuro  
  x <- gsub("[^a-z0-9 ]", " ", x) # 2) troca por espaco tudo que NAO for  
                                #   letra, digito ou espaco  
  x <- gsub("\\s+", " ", x)    # 3) colapsa 2+ espacos em um so  
  trimws(x)                 # 4) remove espacos das pontas  
}  
limpar(brutos[["d1"]])  
  
[1] "recuperacao de informacao ordena documentos por relevancia"
```

O `[[ ]]` (colchete **duplo**) extrai o *conteúdo* sem carregar o nome do elemento.

Entendendo as duas *regex*

"`[^a-z0-9 ]`" — o que **não** queremos

- `[ ]` define um **conjunto** de caracteres.
- `^` *dentro* dos colchetes significa **negação**: “qualquer coisa *exceto*”.
- `a-z0-9` e o espaço são o que **preservamos**; todo o resto (`:`, `,`, `!`, `(`, `)`, `-`, `.`) vira espaço.

"`\\s+`" — espaços sobrando

- `\\s` é “qualquer espaço em branco” (espaço, tabulação, quebra de linha).
- `+` significa “**um ou mais**”. Então `\\s+` casa *sequências* de espaços.

Cuidado: o passo 2 **cria** espaços duplos (a pontuação virou espaço). Por isso o passo 3 vem *depois*.

E para limpar o *corpus* inteiro?

Boa notícia: não precisa de laço. As funções `tolower`, `gsub` e `trimws` são **vetorizadas** — operam sobre o vetor todo de uma vez.

```
limpos <- limpar(brutos)      # os 8 documentos, de uma vez só
head(limpos, 3)

                                d1
"recuperacao de informacao ordena documentos por relevancia"
                                d2
"o modelo de espaco vetorial representa documentos como vetores"
                                d3
"bm25 e um modelo probabilistico de ranqueamento de texto"
```

Os **nomes** (`d1`, `d2...`) são preservados. É assim que trataremos *corpora* de milhares de documentos.

Removendo *stopwords*

Stopwords são palavras muito frequentes e pouco informativas (“de”, “o”, “a”...).

```
stopwords <- c("de","o","a","e","um","por","como","que","da","do")
tok <- function(x) unlist(strsplit(limpar(x), " "))
sem_stop <- function(x) { t <- tok(x); t[!t %in% stopwords] }
sem_stop(brutos[["d2"]])

[1] "modelo"      "espaco"      "vetorial"    "representa"  "documentos"
[6] "vetores"
```


Stemming: reduzir ao radical

Aproxima formas da mesma palavra (*documento*, *documentos*) a uma raiz comum, para que casem na busca.

```
# versao ILUSTRATIVA: corta sufixos comuns do portugues
stem <- function(t) sub("(cao|mento|dade|ais|s)$", "", t)
sapply(c("documentos", "ranqueamento", "relevancia", "modelos"), stem)
```

documentos	ranqueamento	relevancia	modelos
"documento"	"ranquea"	"relevancia"	"modelo"

A regex "(cao|mento|dade|ais|s)\$" diz: “*um destes sufixos, no fim da palavra*” — o | é “ou” e o \$ é “fim”. O sub troca por vazio, isto é, **apaga**.

Na prática: o algoritmo Snowball

Nossa função é um brinquedo. Em produção usa-se o **Snowball** (evolução do algoritmo de Porter, 1980), que é *baseado em regras*, não em dicionário.

Como funciona

- ① Divide a palavra em regiões (o que vem depois da primeira vogal-consoante).
- ② Aplica **séries ordenadas** de regras de corte: plural, feminino, aumentativo, verbal, adverbial. . .
- ③ Cada regra só dispara se o radical restante for **longo o bastante** — isso evita destruir palavras curtas.

Existe uma versão específica para o português (e o **RSLP**, algoritmo brasileiro de Orengo & Huyck, 2001).

Usando o SnowballC (é um pacote)

Não vem com o R: é um **pacote** que precisa ser instalado **uma vez**.

```
install.packages("SnowballC")      # so na primeira vez
library(SnowballC)

wordStem(c("documentos", "documento", "documentacao"),
         language = "portuguese")
#> [1] "document" "document" "document"
```

Atenção: *stemming* é destrutivo

O radical costuma **não ser** uma palavra real (“document”), e formas diferentes podem colapsar demais. Ganha-se *recall*, perde-se *precisão* — é um **trade-off**, não um ganho puro.

Tokenização *subword*: o problema do vocabulário

Até aqui, cada **palavra** é um token. Isso tem três defeitos:

- **Palavras desconhecidas** (*OOV, out-of-vocabulary*): “Bertioga” não estava no treino → o modelo não sabe o que fazer.
- **Morfologia**: “praia”, “praias”, “praiano” viram tokens **independentes**, sem relação entre si.
- **Vocabulário explode**: milhões de formas distintas em textos reais.

A saída

Quebrar palavras em **pedaços menores** (*subwords*): um vocabulário pequeno e **fechado** representa *qualquer* palavra, mesmo nunca vista.

O BPE é um algoritmo de tokenização

Situando

Sim: o BPE (*Byte Pair Encoding*) é uma forma de **tokenizar** — decidir quais são as unidades do texto. Ele apenas escolhe unidades *diferentes* das que usávamos.

Tokenização	“documentos” vira...
por caractere	d o c u m e n t o s
por palavra (Aulas 01–06)	documentos
por subword (BPE)	document + os

Caractere: vocabulário mínimo, sequências longíssimas. Palavra: sequências curtas, vocabulário infinito. **Subword é o meio-termo.**

Como o BPE aprende os pedaços

- 1 Começa com o vocabulário mínimo: os **caracteres**.
- 2 Conta todos os **pares vizinhos** no *corpus*.
- 3 **Funde** o par mais frequente num símbolo novo.
- 4 Repete até atingir o tamanho de vocabulário desejado.

O resultado é curioso

Os pedaços mais frequentes acabam sendo **radicais e sufixos** — o algoritmo “descobre” morfologia **sem que ninguém a ensine**. Vamos ver isso acontecer em três passos.

BPE passo 1: começar pelos caracteres

Nosso mini-*corpus*: quatro palavras, cada uma com **quantas vezes** aparece.

```
freq <- c(praias = 5, ilhas = 4, ruas = 3, praia = 2)

# quebra cada palavra em caracteres: "praias" -> "p" "r" "a" "i" "a" "s"
simbolos <- lapply(names(freq), function(p) strsplit(p, "")[[1]])
names(simbolos) <- names(freq) # devolve os nomes das palavras
simbolos[["praias"]]
```

```
[1] "p" "r" "a" "i" "a" "s"
```

O vocabulário inicial é só o alfabeto. A partir daqui, o algoritmo vai **juntando** pedaços.

BPE passo 2: contar os pares vizinhos

```
contar_pares <- function(simb, freq) {  
  pares <- integer(0)                                # placar vazio  
  for (p in names(simb)) {                            # para cada palavra...  
    s <- simb[[p]]                                     # ...pegue seus simbolos  
    for (i in seq_len(length(s) - 1)) {               # ...e cada par vizinho  
      k <- paste0(s[i], s[i + 1])                    # nome do par: "pr", "ra"...  
      pares[k] <- ifelse(is.na(pares[k]), 0, pares[k]) + freq[[p]]  
    }                                                  # soma a FREQUENCIA da palavra  
  }  
  sort(pares, decreasing = TRUE)                     # do mais comum ao mais raro  
}  
head(contar_pares(simbolos, freq), 4)  
  
as pr ra ai  
12 7 7 7
```


par	contagem	de onde vem
as	12	praias (5) + ilhas (4) + ruas (3)
pr	7	praias (5) + praia (2)
ra	7	praias (5) + praia (2)
ai	7	praias (5) + praia (2)

- Cada par é contado **ponderado** pela frequência da palavra em que aparece.
- as vence porque aparece em **três** palavras diferentes; pr só em duas.

BPE passo 3: fundir o par vencedor

Para enxergar a fusão, escrevemos cada palavra com os símbolos **separados por espaço**. Fundir vira então um gsub — o mesmo da Aula 00.

```
palavras <- c(praias = "p r a i a s", ilhas = "i l h a s",  
              ruas   = "r u a s",      praia = "p r a i a")  
  
gsub("a s", "as", palavras)  # "a s" (dois simbolos) -> "as" (um so)  
  
      praias      ilhas      ruas      praia  
"p r a i a s"  "i l h a s"  "r u a s"  "p r a i a"
```

Agora as é **um símbolo único** do vocabulário. O algoritmo repetiria o processo: contar pares de novo, fundir o próximo vencedor, e assim por diante.

O que aconteceu nesses três passos

palavra	antes	depois da 1ª fusão
praias	p r a i a s	p r a i a s
ilhas	i l h a s	i l h a s
ruas	r u a s	r u a s
praia	p r a i a	p r a i a

O achado

O BPE descobriu sozinho o sufixo “-as” — a marca de plural do português. E repare: praia (singular) **não** foi afetada. Ninguém programou gramática: isso emergiu da **frequência**.

Método	Como escolhe o merge	Usado em
BPE	par mais frequente	GPT, Llama, RoBERTa
WordPiece	par que mais aumenta a verossimilhança	BERT
SentencePiece	trata o espaço como símbolo; não precisa de pré-tokenização	T5, mT5, Llama

SentencePiece é essencial para idiomas **sem espaços** (japonês, chinês) — e útil em qualquer língua.

Quando usar cada tokenização

Palavra inteira

- Índice invertido e **BM25**
- Interpretável: o termo é legível
- O que usamos nas Aulas 01–06

Subword

- **Embeddings** e modelos neurais
- Sem palavras desconhecidas
- O que veremos nas Aulas 07 e 14

Guarde isto

Não existe “a” tokenização correta: ela é uma **decisão de projeto** que muda o vocabulário, o índice e o desempenho final.

O problema: buscar lendo tudo é inviável

A busca ingênua percorre **todos** os documentos e procura o termo em cada um:

Coleção	Documentos	Leituras por consulta
nosso exemplo	8	8
Wikipédia (pt) web	1,1 milhão bilhões	1,1 milhão bilhões

A ideia

E se, em vez de ler os documentos a cada busca, a gente **pré-calculasse** uma vez onde cada palavra está?

A analogia: o índice remissivo de um livro

Índice **direto**

O livro em si:

página → palavras que contém.

Para achar “entropia”, você leria **o livro inteiro**.

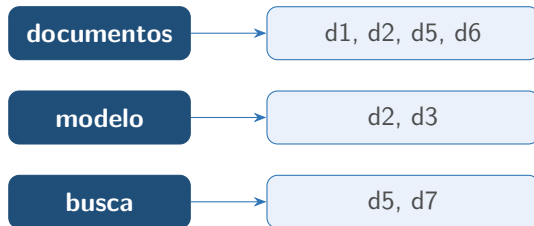
Índice **invertido**

O índice remissivo do fim:

palavra → páginas onde aparece.

“entropia ... 42, 87” — você vai **direto**.

“Invertido” é exatamente isso: **inverte-se** a relação. Em vez de documento → termos, guardamos termo → documentos.



Para cada **termo**, guardamos a *lista de postagens*: em quais documentos ele aparece. A busca vira **consulta ao dicionário**, não varredura de todos os textos.

- **Dicionário:** a lista de todos os termos distintos (o vocabulário).
É por onde a busca *entra* — procurar aqui é praticamente instantâneo.
- **Lista de postagens** (*postings list*): para cada termo, os documentos em que ele aparece.
É o que a busca *devolve*.

Em R

Usaremos uma `list()`: os **nomes** da lista são o dicionário, e cada **elemento** é a lista de postagens daquele termo. Ou seja, `postings[["busca"]]` devolve `"d5d7"`.

Em sistemas reais, a postagem guarda mais: a frequência do termo e até as **posições** em que ele ocorre (para buscar frases exatas).

Construindo o índice em R

```
prep <- function(x) sem_stop(x)  # limpa + tokeniza + tira stopwords
postings <- list()               # começa vazio

for (d in names(brutos)) {      # 1) para cada documento...
  for (termo in unique(prepare(brutos[[d]]))) { # 2) cada termo DISTINTO
    postings[[termo]] <- c(postings[[termo]], d) # 3) anexa o doc
  }
}

postings[["documentos"]]

[1] "d1" "d2" "d5" "d6"

postings[["modelo"]]

[1] "d2" "d3"
```

O laço, linha a linha

- ① `for (d in names(brutos))` — percorre os documentos **uma única vez**. Este é o custo de construção, pago **antes** de qualquer busca.
- ② `unique(prepare(...))` — o `unique` evita repetir o documento na lista caso o termo apareça duas vezes nele.
- ③ `postings[[termo]] <- c(postings[[termo]], d)` — **anexa** o documento à lista daquele termo.

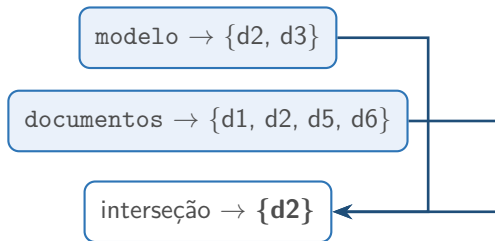
Se o termo é novo, `postings[[termo]]` é `NULL` e o `c()` simplesmente cria a lista com um elemento.

A inversão acontece aqui

Note a direção: lemos por **documento**, mas gravamos por **termo**. É literalmente a inversão da relação.

Buscando: a consulta vira operação de conjuntos

Para a consulta "modelo documentos", queremos os documentos que têm **os dois** termos:



- Consulta “**E**” (AND) → **interseção** das listas.
- Consulta “**OU**” (OR) → **união** das listas.

Não lemos **nenhum** documento — apenas cruzamos duas listas curtas.

Busca pelo índice (interseção)

```
busca_AND <- function(consulta) {  
  termos <- prep(consulta)           # mesma limpeza usada na indexacao!  
  Reduce(intersect, postings[termos]) # intersecta as listas, 2 a 2  
}  
busca_AND("modelo documentos")  
  
[1] "d2"  
  
busca_AND("busca documentos")  
  
[1] "d5"
```

Reduce aplica intersect acumulando: com 3 termos, faz `intersect(intersect(A,B),C)`.

Regra de ouro: a consulta precisa passar pelo *mesmo* pré-processamento dos documentos — senão “Documentos” não acha “documentos”.

```
length(postings) # termos indexados
```

```
[1] 37
```

```
sort(lengths(postings), decreasing = TRUE)[1:4]
```

documentos	recuperacao	relevancia	modelo
4	2	2	2

Termos com listas maiores aparecem em mais documentos — exatamente os de menor IDF.

Ponte para a próxima fase

- Já temos **texto limpo** e um **índice invertido** eficiente.
- Falta combinar o índice com **pesos** para ranquear melhor que TF-IDF puro.
- Na Fase 2 entra o **BM25**: um modelo **probabilístico** que ajusta frequência e tamanho.

Resumo das Aulas 01–03

Texto → termos → índice → vetores → ranking. A base do motor está pronta.

Tarefa

- ① Montar um índice invertido do *corpus* de 8 documentos.
- ② Adicionar *stemming* com `SnowballC::wordStem(..., "portuguese")`.
- ③ Implementar `busca_AND` e `busca_OR` e comparar os resultados.

Próxima fase

- Modelo probabilístico: BM25
- Avaliação ($P@k$, MAP, nDCG)

Leitura

- Manning et al., *IIR* — cap. 1–2
- Baeza-Yates & Ribeiro-Neto — cap. 4

Obrigado!

Dúvidas? Vamos ao R.
